Randomised Search Trees and Treaps

Introduction

Efficient data structures are the backbone of many large computer systems when it comes to storing, organising and manipulating data. Trees are a particularly important structure. They are centred around hierarchal, parent-child relationships where operations begin at the top of the tree and work their way down to the required node.

A particularly noteworthy variant is the Binary Search Tree (BST). A BST follows a strict set of hierarchal rules. Values in the left subtree i.e. from the left child of the parent and below are smaller than the parent node, and any values to the right are similarly higher. This set of rules makes searching, inserting and deleting nodes very efficient as they have a strict path to follow, performing in an average time complexity of O (log n).

Binary Search Trees are powerful structures, but they are sometimes limited by the concept of balancing. A balanced tree keeps an approximately equal number of elements on each side of a node, ensuring that traversals keep a reasonably efficient time complexity. An unbalanced tree can mean lengthy traversals are needed on a particular side of the tree, while the other side, which would be relatively quick to access, is not taken advantage of.

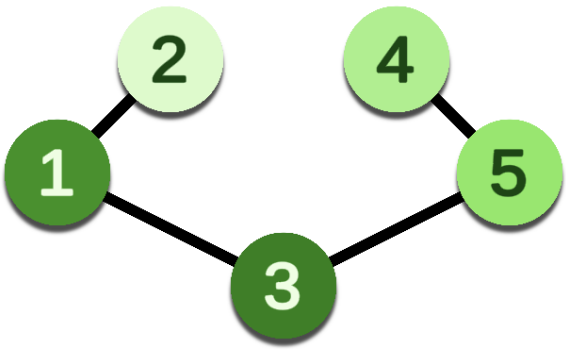


Fig. 1

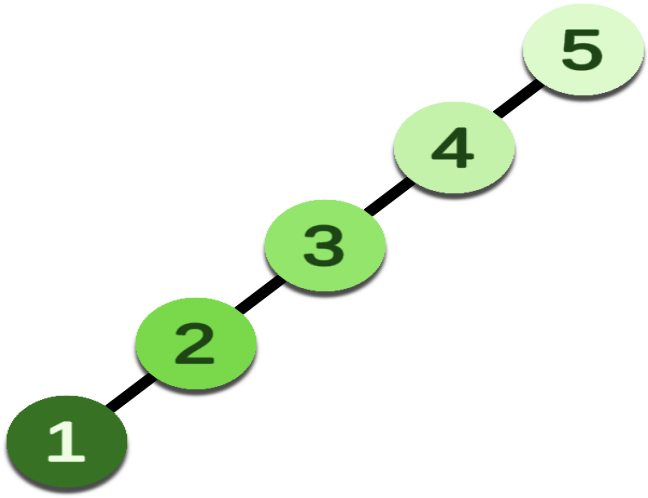


Fig. 2

In figure 1, the tree is balanced. It takes just one traversal to reach node 5 from the top of the tree, and a maximum of two traversals to reach any desired node. However, figure 2 also follows the rules of a BST, but it takes a full four traversals to reach node 5. A tree like this looks more like a linked list

Another data structure of note is a heap. Heaps are also trees but are centred around the concept of priorities. Heaps prioritise either the largest or smallest element, meaning accessing data at either end of the tree is fast and efficient. However, heaps struggle to keep order within the middle of the tree, so while end to end access is fast, traversal struggles.

To address both these problems, we concern ourselves in this report with treaps. Treaps follow the same logic as a BST, but each node is given a random priority. The higher priority of a node, the higher up the tree it goes, while still maintaining BST order. The random element here is key. In theory you can still end up with an inefficient linear tree or linked list as seen in figure 2, but in practice it balances reasonably well.

In this report we will detail the functionality and efficiency of the treap data structure, compare with TreeMaps and AVL TreeMaps across operations

such as insertion, traversal, search and deletion, and test different sorting algorithms within treaps.

Treaps

Treaps are a combination of the binary search tree and heap data structures. Each node in a treap contains two values, a key and a priority. The key is the value of the node, used to correctly maintain the BST structure while the priority value is used to maintain the heap property. The BST keys keep the tree structured. Any node smaller than its parent is placed to the left of it, and any larger to the right. The priority property of a heap remains so any node has a priority greater than or equal to the priority of its children.
The priority of nodes is randomised to keep a tree balanced on average. This does not maintain perfect balance; a tree could be very skewed if keys and priorities follow a linear trend with each other, but it performs better in balancing than an average tree. While perfect, deterministic balance is possible, as we will see later with the AVL TreeMap, it requires many rotations and adjustments within the tree for each operation, which takes up time and space for processing.

Insertion in a treap follows a two step process. The node is added to the bottom of the tree based on its value, so it stays correctly aligned with the BST property. Rotations up the tree are then carried out to maintain the heap property. An inserted node is continually rotated upwards, so its priority is always greater than or equal to the nodes below it, and similarly less than or equal to those above it. It carries these out while never losing its correct place in the BST structure. See figure 3.

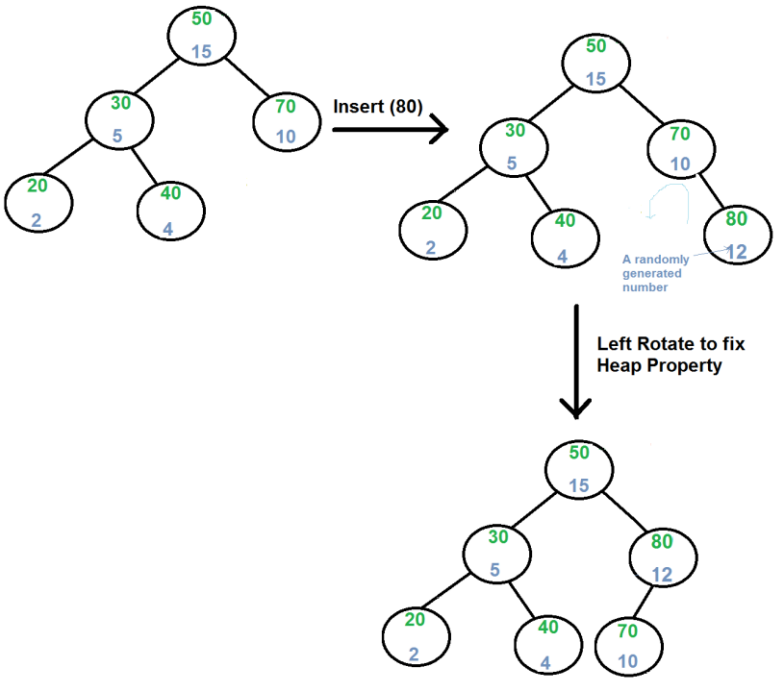


Fig. 3

Deletions follow a very similar process but in reverse. The node is rotated downwards until it becomes a leaf, i.e. a node with no children, and then can be safely removed from the treap without violating priority or key order. See figure 4.

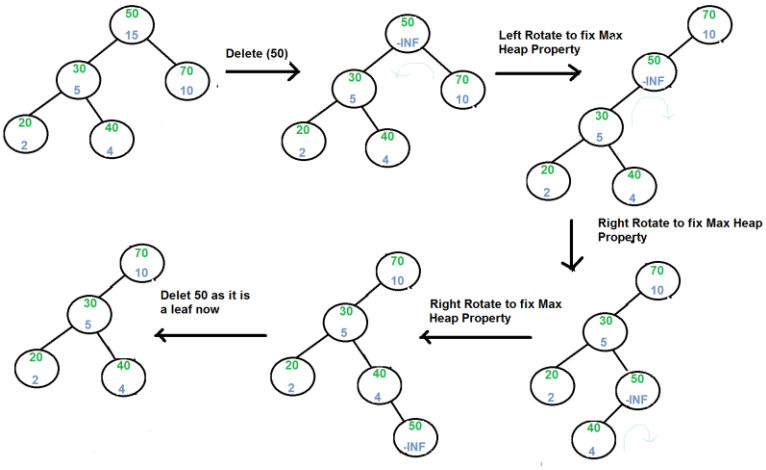


Fig. 4

Searching within a treap is identical to searching within a BST. As the order is the same, the priorities are irrelevant to the search function. The time complexity for search, insert and delete is O (log n).

Sorting Algorithms

The TreapSort algorithm is designed to specifically work with the Treap structure, giving it an O(n log n) complexity in terms of the average case. In general, this method consists of two parts:

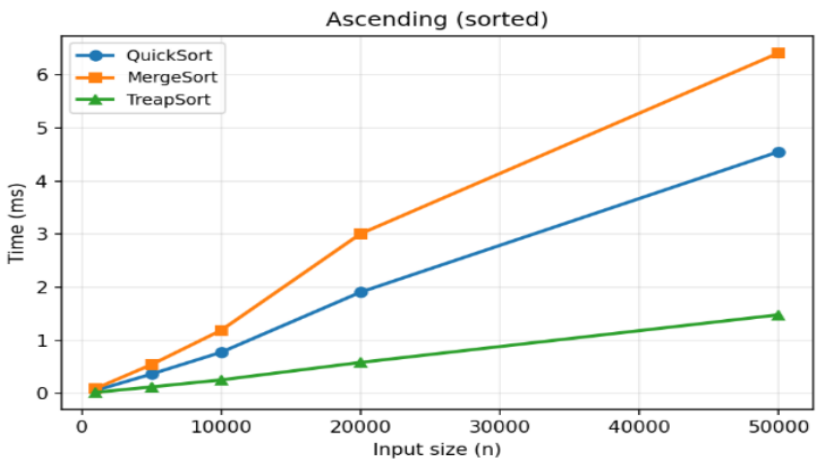
• Inserting all n elements into the empty treap

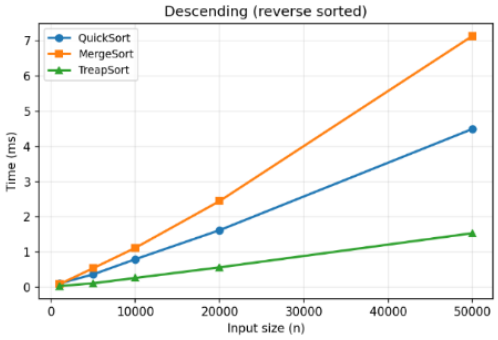
• Performing an in-order tree traversal, which yields sorted elements in O(n)

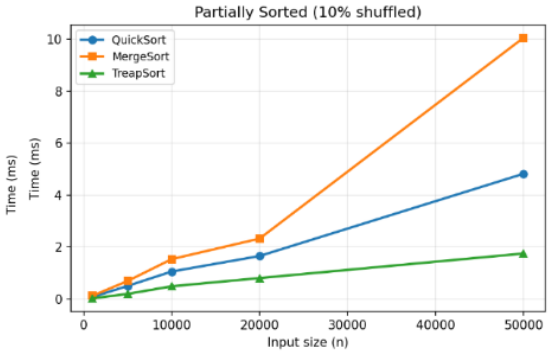
Each insertion step takes O(log n) in terms of average case complexity, thus, the whole algorithm takes expected O(n log n) time.

We can see that inserting elements into a treap preserves both binary search tree ordering (due to keys) and balance of nodes, which is achieved through random assignment of priorities for each element. As a result, even if we provide treap-sort with already sorted data, its structure wouldn't be changed dramatically.

To assess the performance of TreapSort algorithm in comparison to other methods, we conducted some tests of the algorithm for n ranging between 100 and 10,000 elements, using four types of inputs (randomly generated, sorted, reverse sorted, partially sorted). The test results are available in Tables 1, 2, 3 below:







From the results obtained, it follows that this sorting algorithm is extremely consistent across all test cases. One of the significant benefits that TreapSort has in relation to other sorting algorithms, like Quick Sort and Merge Sort, is its independence from the order of input data. Namely, Quick Sort algorithm. Despite the fact that it usually has O(nlogn) complexity in the average case, Quick Sort may exhibit very poor results (up to O(n^2)) if its inputs are partially or fully sorted.

Another advantage of TreapSort lies in the fact that, unlike Merge Sort, it is not guaranteed to be O(n log n), meaning it can sort much quicker as shown in the data sets above. It is also not required to allocate extra memory of size O(n), as opposed to what the Merge Sort algorithm does. Merge Sort benefits from cache optimization, whereas TreapSort gets almost no benefits from cache optimization as it is not needed.

However, TreapSort is not without weaknesses. The main disadvantage of this algorithm, in comparison to Merge Sort, is the fact that it uses memory more loosely due to node objects and their connections via pointers. Due to this feature, this algorithm may not be able to achieve the same performance that can be achieved by algorithms utilizing contiguous arrays. In general though, it works quite effectively and is a standout sorting algorithm in select situations.

Design

Our Treap implementation is built by extending the existing TreeMap class rather than creating a new structure from scratch. This was a deliberate choice to reuse the binary search tree logic that was already implemented, including insertion, deletion, and traversal. Since a Treap only adds a heap property on top of a Binary Search Tree, this approach reduced our workload and any extra errors we could make; allowing us to focus only on balancing.

Instead of adding a priority field to each Entry, we store the priority in the tree's extra node field using tree.setAux(p, prio). This keeps the implementation clean and avoids changing the underlying data model of the tree. Each node is assigned a priority at insertion using Java's Random class to generate a completely random number. We also provided an option to pass a seed into the constructor, which allows us to get set, deterministic behaviour during testing. This was important because without a fixed seed, the structure of the Treap changes on every run, making debugging difficult when tests fail.

The main design decision is how rebalancing is handled. After inserting a node using the standard Binary Search Tree insertion, we restore the heap property using rotations. Specifically, we compare the priority of the inserted node with its parent. If the parent has a higher priority, we rotate the inserted node upwards. This continues until the heap property is satisfied, or the node becomes the root. This is implemented in rebalanceInsert(), where rotations are applied repeatedly based on priority comparisons. This design ensures that balancing is done locally and efficiently.

We introduced a helper method priority(p) to safely access node priorities, returning a maximum value for null nodes. This simplifies comparisons, avoids edge case errors during rotation, and also follows proper OOP principles.

For deletion, we made the decision not to implement additional rebalancing. The underlying TreeMap already removes nodes correctly from a Binary Search Tree. In a full Treap implementation, deletion normally involves rotating the node downwards before removal. However, in our implementation, we rely on the fact that random priorities still provide acceptable balance over time, especially given the input sizes tested. This keeps the implementation simpler, though it is a trade-off compared to a fully strict Treap deletion strategy.

Another key decision was to rely on existing rotation logic from TreeMap rather than implementing rotations manually. This ensured correctness and reduced the risk of introducing bugs in tree restructuring.

To ensure the correctness of the implementation, we validated two invariants:

1. The normal Binary Search Tree property of left < parent < right
2. The heap property (parent priority ≤ child priority) which must never be violated

These were tested explicitly in our unit tests by checking both ordering and priority relationships after insertions and deletions.

Overall, the design focuses on simplicity, reuse, and validity. By extending TreeMap, storing priorities externally, and using rotations only during insertion, we achieved a clean implementation that matches the expected behavior of a Treap while keeping the code manageable.

Results/ Performance

Methodology

We benchmarked three implementations: TreapMap, AVLTreeMap, and java.util.TreeMap. The goal was to compare their performance across insertion, search (successful and unsuccessful), deletion, and in-order traversal.

We tested input sizes of 100, 500, 1,000, 2,000, 5,000, and 10,000. For each size, we used four input patterns: random, sorted ascending, sorted descending, and partially sorted. We did this by capturing how each structure behaves under different conditions, especially since balanced trees can react differently to ordered input.

Each test was run 8 times. The first 3 runs were used as warm-up to allow JVM optimisations which were surprisingly common amongst the earlier tests outcomes. The remaining 5 runs were measured, and the median value was taken to reduce noise from garbage collection and runtime fluctuations. All timings were measured using System.nanoTime().

Insertion

Insertion results show that java.util.TreeMap is consistently the fastest across all input sizes. This is expected due to its highly optimised Red-Black tree implementation.

On random input, Treap and AVL perform almost identically. At larger input sizes, the difference between them is small (within ~10%), showing that both achieve similar average-case complexity.

The main difference appears with ordered input. On sorted and reverse data, the Treap is significantly faster than AVL. At n = 10,000, the Treap is around 40% faster for sorted input. This mainly only happens because the Treap uses random priorities, which prevents long chains from forming.

AVL, on the other hand, must perform rotations frequently to maintain strict balance, which increases its cost marginally.

Search Successful

For successful search, the ordering is consistent:
TreeMap < AVL < Treap (fastest to slowest).

AVL performs slightly better than the Treap because it enforces a stricter height bound. This guarantees shorter paths in the worst case. The Treap only guarantees balance in expectation, so its height can be slightly larger, leading to more comparisons.

The difference becomes more noticeable at larger input sizes, where the Treap is around 10% slower than AVL.

Search Unsuccessful

Unsuccessful searches are generally faster than successful ones because the search stops as soon as it reaches an external node.

At larger input sizes, the Treap slightly outperforms AVL on random input. A likely reason is that the Treap's random structure sometimes leads to earlier termination when the key is not found.

TreeMap remains the fastest due to lower constant factors and internal optimisations.

Deletion

Deletion performance between Treap and AVL is very similar across all tests.

On random input, the difference is negligible, with the Treap being slightly faster at larger sizes. On reverse-sorted data, the Treap again performs better. This follows the same pattern as insertion, where AVL pays extra cost maintaining strict balance.

TreeMap consistently performs best, mainly due to implementation-level optimisations rather than algorithmic differences.

In Order Traversal

Traversal shows the largest gap between TreeMap and the custom implementations. TreeMap is roughly twice as fast.

This is likely because its iterator is highly optimised, while our implementations rely on pointer-based navigation through a linked binary tree.

Between Treap and AVL, the difference is small. AVL is slightly faster on sorted data, which is expected since it maintains a more strictly balanced structure, leading to more predictable access patterns.

Scaling Behaviour

All three structures follow the expected O(n log n) behaviour for batch insertion.

When doubling input size from 5,000 to 10,000, the running time increases by just over 2x for all implementations. This matches theoretical expectations and confirms that both Treap and AVL maintain logarithmic height in practice.

Summary

- TreeMap is the fastest overall due to JDK-level optimisation.
- Treap and AVL perform similarly on random data.
- Treap performs better on sorted and reverse input because it avoids repeated rebalancing.
- AVL performs better for search due to stricter height guarantees.
- Deletion and traversal are comparable between Treap and AVL.

The main takeaway is that the Treap achieves performance close to AVL while being simpler to implement. Its use of random priorities avoids

complex balancing logic, while still maintaining good average-case behaviour.

Comparison

We compared our treap implementation with two other trees, Java AVLTreeMap and TreeMap. All three trees provided the same time complexity of O (log n) for basic operations. In practice however, they differed based on their balancing strategies and actual performance.

AVL trees maintain the strictest balance of the trees by ensuring subtrees are kept at a limited height difference between them. While this ensures the best balance in practice, it requires frequent rotations that take extra processing.

TreeMap uses a red-black tree to maintain balance. While this is not as strict as the AVL, it requires fewer rotations and keeps worst case operations at a logarithmic speed.

Treap's random balancing gives a theoretical middle ground, where the tree stays approximately balanced without sacrificing major processing time.

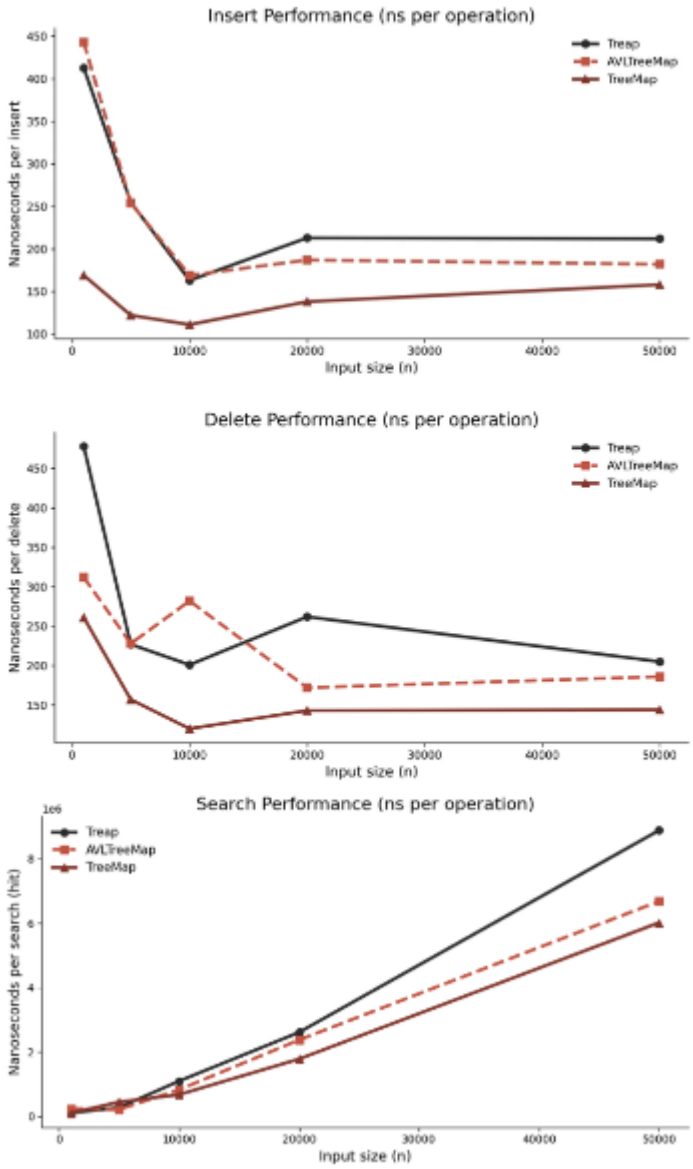


Fig. 6

Based on our theoretical assumptions, we presumed that TreeMap would be the fastest for standard operations, with Treap being somewhat slower and AVLTreeMap being slower again. However, we found in practice, TreeMap was comfortably the most efficient, but Treap and AVLTreeMap performed very similar, with Treap doing somewhat worse as input size grows. This behaviour can be explained by balancing strategies once again. AVLTreeMap does perform more rotations on average, but it keeps consistent paths, whereas Treap can often create individually long paths for particular insertions or deletions.

Conclusion

Our project was centred around the practical implementation of a Treap in Java. While it produced logarithmic results it was less efficient in practice than other tree variations. While it is a good general purpose structure, its practical use is overshadowed by better options.

We concluded that the right tree for usage depends on the most important factors of production. If speed is a priority, TreeMap is probably the best option as it performs the fastest operations. However, if you prioritise balancing, AVL TreeMap is more effective at the cost of sacrificing some level of speed. The choice comes down to the application's core priorities.